

8



Módulo 08 Conceitos de criptografia

Aula 03 *Funções Hash*

Introdução —

Nesta aula, você conhecerá os seguintes assuntos:

- Algoritmos poderosos e versáteis, amplamente utilizados em:
- Criptografia.
- Verificação de integridade de arquivos.
- Autenticação de dados e muito mais.

Aula 03

Funções *Hash*

Características das Funções *Hash* —

```

style="background-image:url('/pix/samples/bgl.gif');
<!--Fixed Width 2 Blue</title>
<style type="text/css">
<div style="background-image:url('/pix/samples/bgl.gif');background . text- todoitem ;
height . text - :200px;"><p>The image can be tiled across the background,
while the text runs across the top.</p> </div>

/* Logo */
<body style="background-color:yellowgreen;color:white;">
<html> <.todolistid = data.todoid;
<html> <.todolistid = data.todoid;

/* Header */
<div style="background:#eee; }
#header { margin:0 auto; padding:10px; width:970px;background:#fff)
#header-inner { margin:0 auto; padding:10px; width:970px;background:#fff)

/* Feature */
<div style="background-image:url('/pix/samples/bgl.gif');background . text- todoitem ;
height . text - :200px;"><p>The image can be tiled across the background,
while the text runs across the top.</p> </div>

/* Menu */
<p>You can make <span style="font-style:italic">some</span> the HTML 'span' tag.
<p>You can hold <span style="float:left;
#top-nav ul li { margin:0; padding:0 0 8px; float:left;
#top-nav ul li { display:block; margin:0; padding:8px 20px; color:blue; text-decoration:none;
#top-nav ul li { background-color:yellowgreen;color:white;"> { color:#d3d3d3;

<body style="background-color:yellowgreen;color:white;"> the HTML 'span' tag.
<div style="background-image:url('/pix/samples/bgl.gif');background . text- todoitem ;
height . text - :200px;"><p>The image can be tiled across the background,
while the text runs across the top.</p> </div>

/* Content */
<p>You can make <span style="font-style:italic">some</span> of your text using the HTML tag.</p>
<p>You can hold <span style="float:left;
<div style="background-image:url('/pix/samples/bgl.gif');background . text- todoitem ;
height . text - :200px;"><p>The image can be tiled across the background,
while the text runs across the top.</p> </div>
<body style="background-color:yellowgreen;color:white;">
#content #sidebar .widget ul li { margin:0;
#content #sidebar .widget ul li { padding:4px 0; width:185px;
#content #sidebar .widget ul li { color:blue; text-decoration:none; margin-left:-16px; padding:4px 8px 4px 16px;
#content #sidebar .widget ul li { background-color:yellowgreen;color:white;">
while the text runs across the top.</p> </div>

/* Footer */
#footer { background:#eee;
#footer-inner { margin:auto; text-align:center; padding:12px; width:970px;
<html> <p style="font-weight:bold;"
>HTML form code is done using CSS.</p>
<html> <.todolistid = data.todoid;

```

- Algoritmo que recebe dados de entrada e produz uma sequência de caracteres ou valores de tamanho fixo, chamados de "*hash*" ou resumo.
- É rápida de calcular e determinística, ou seja, a mesma entrada sempre produzirá o mesmo *hash*.

ALGORÍTIMO DE HASH



Fonte: Macoratti.
Disponível em: <https://macoratti.net/16/09/net_cripto2.htm>.
Acesso em: 20 mar. 2024.

FUNÇÃO HASH

- Objetivos:

- Mapear um conjunto de dados de tamanho arbitrário para um valor de *hash* de tamanho fixo.
- Representa de forma única e compacta os dados originais.



FUNÇÃO *HASH*

- Usado em:

- Criptografia.
- Verificação de integridade de dados.
- Autenticação.
- Indexação de informações.





Fonte: Mitsloan Management Review, 2023.
Disponível em: <<https://www.mitsloanreview.com.br/post/leis-de-protecao-de-dados-e-blockchain-solucoes-da-propria-tecnologia-parte>>.
Acesso em: 20 mar. 2024.

FUNÇÃO *HASH*

- Uma função *hash* de qualidade deve ser capaz de produzir valores de *hash* únicos para diferentes conjuntos de dados.
- Desempenha um papel importante na proteção de senhas e na verificação da integridade dos dados transmitidos.



RESISTÊNCIA A TENTATIVAS DE INVERSÃO

- É uma propriedade fundamental de uma função *hash*.
- Refere-se à dificuldade de recuperar ou reverter os dados originais a partir do valor de *hash* gerado por essa função.
- Encontrar a entrada original a partir do valor de *hash* deve ser computacionalmente difícil e impraticável.



Fonte: Crypto ID, 2022.
Disponível em: <<https://cryptoid.com.br/criptografia-identificacao-digital-id-biometria/the-paper-that-can-change-the-foundations-of-all-blockchain-cryptography-jesus-rodrigues/>>.
Acesso em: 20 mar. 2024.

RESISTÊNCIA A TENTATIVAS DE INVERSÃO

- É essencial para garantir a segurança de diversas aplicações.
- Exemplo:
 - Quando uma senha é armazenada em um sistema, ela não é armazenada diretamente, mas sim o seu valor de *hash*.
 - Quando um usuário tenta fazer login, a senha inserida é novamente convertida em um valor de *hash*.
 - Esse *hash* é comparado com o valor armazenado anteriormente.
 - Se ambos os valores de *hash* coincidirem, a senha é considerada correta.



Fonte: Crypto ID, 2022.

Disponível em: <<https://cryptoid.com.br/criptografia-identificacao-digital-id-biometria/the-paper-that-can-change-the-foundations-of-all-blockchain-cryptography-jesus-rodrigues/>>.

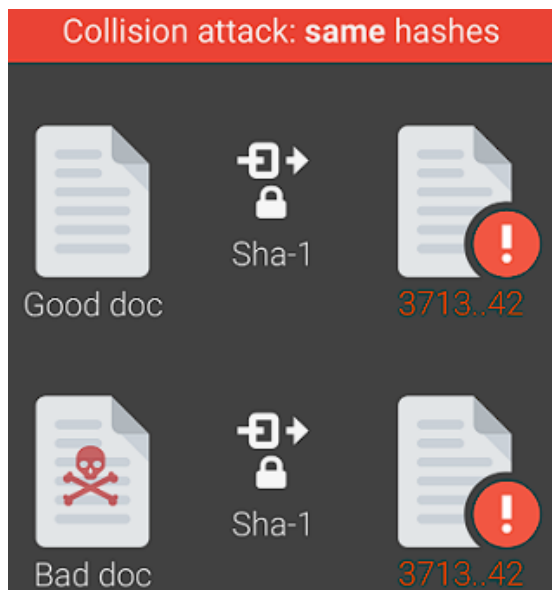
Acessado em: 20 mar. 2024.

RESISTÊNCIA A TENTATIVAS DE INVERSÃO

- Garante que mesmo que um atacante obtenha acesso aos *hash* armazenados, ele não poderá determinar a senha original correspondente.
- O atacante precisaria testar diferentes combinações de entrada e calcular os valores de *hash* correspondentes até encontrar uma colisão válida.



Fonte: Crypto ID, 2022.
Disponível em: <<https://cryptoid.com.br/criptografia-identificacao-digital-id-biometria/the-paper-that-can-change-the-foundations-of-all-blockchain-cryptography-jesus-rodrigues/>>.
Acesso em: 20 mar. 2024.



Fonte: Security Affairs, 2017.
Disponível em:
<<https://securityaffairs.com/56608/hacking/shattered-attack.html>>.
Acesso em: 20 mar. 2024.

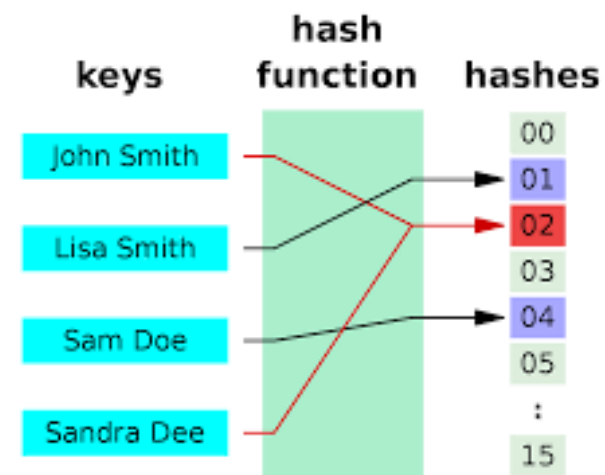
COLISÕES

- Ocorrem quando dois conjuntos de dados distintos produzem o mesmo valor de *hash*.
- É quando duas entradas diferentes são mapeadas para o mesmo resultado de *hash*.
- É indesejável, pois uma das principais propriedades de uma função *hash* é a unicidade.



COLISÕES

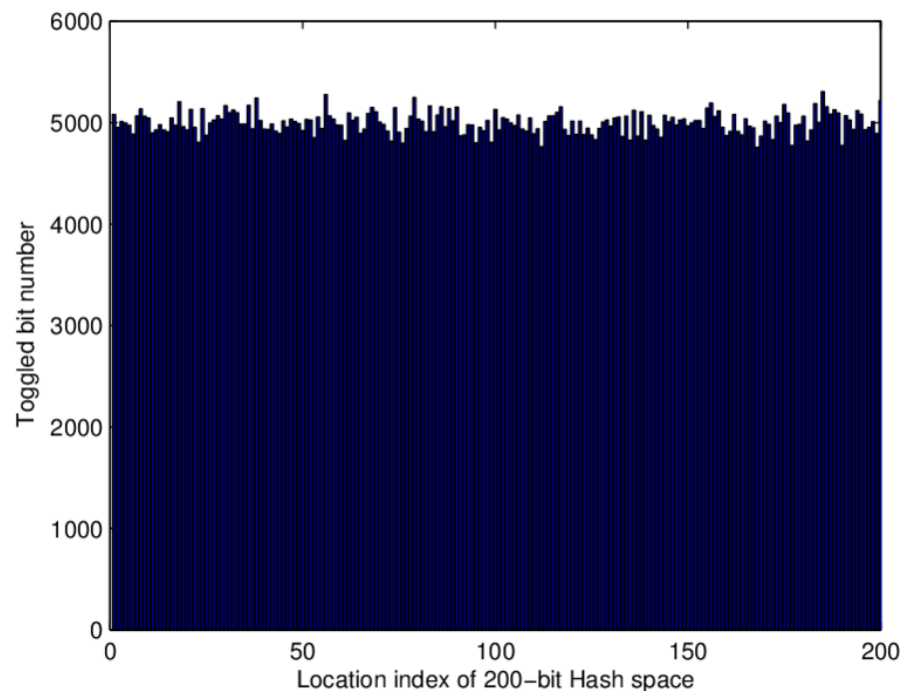
- Podem ser exploradas para falsificar informações ou comprometer a integridade dos dados.
- É fundamental utilizar funções *hash* que apresentem uma resistência adequada a colisões.
- Os algoritmos utilizados devem ser cuidadosamente projetados e analisados para minimizar as chances de colisões.



Fonte: Quora, 2020.

Disponível em: <<https://pt.quora.com/Como-funciona-a-implementa%C3%A7%C3%A3o-de-tabelas-Hash>>.

Acesso em: 20 mar. 2024.



Fonte: Research Gate, 2018.

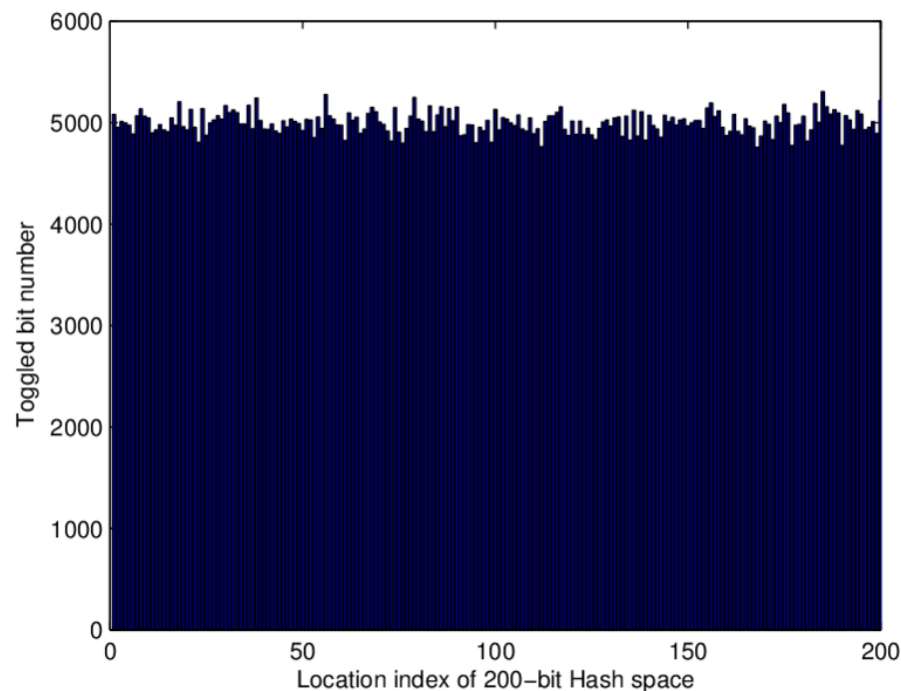
Disponível em:

<https://www.researchgate.net/publication/318670787_Controlled_Alternate_Quantum_Walks_based_Quantum_Hash_Function>.

Acesso em: 20 mar. 2024.

DISTRIBUIÇÃO UNIFORME

- Refere-se à maneira como os valores de *hash* são distribuídos no espaço de saída da função.
- Garante que todos os possíveis valores de *hash* tenham a mesma probabilidade de serem gerados.
- É desejada porque evita a ocorrência de colisões excessivas.



Fonte: Research Gate, 2018.
Disponível em:
<https://www.researchgate.net/publication/318670787_Controlled_Alternate_Quantum_Walks_based_Quantum_Hash_Function>.
Acesso em: 20 mar. 2024.

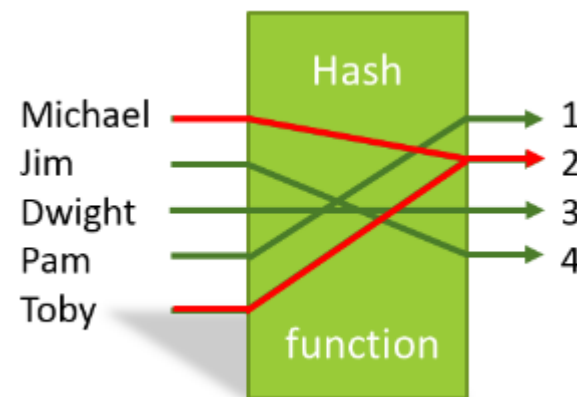
DISTRIBUIÇÃO UNIFORME

- Garante que, independentemente dos dados de entrada, espalhe os valores de *hash* de forma equilibrada em todo o espaço de saída.
- Se a distribuição dos valores de *hash* não for uniforme:
 - Pode haver agrupamentos.
 - Regiões no espaço de saída com uma maior concentração de valores de *hash*.
 - Aumenta a probabilidade de colisões.



UNICIDADE EM FUNÇÕES *HASH*

- Garante que cada conjunto de dados de entrada produza um valor de hash único.
- Dois conjuntos de dados diferentes não devem gerar o mesmo valor de *hash*.
- Permite identificar e distinguir de forma única conjuntos de dados específicos por meio de seus valores de *hash* correspondentes.



Fonte: Stack Overflow, 2022.

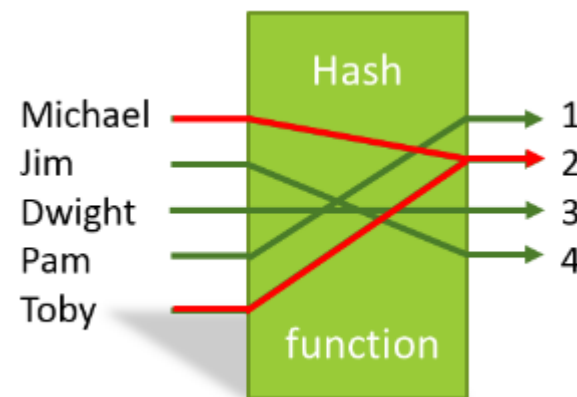
Disponível em:

<<https://stackoverflow.com/questions/39375600/what-hash-function-produces-the-maximum-number-of-collisions-when-hashing-n-keys>>.

Acesso em: 20 mar. 2024.

UNICIDADE EM FUNÇÕES HASH

- É essencial em várias aplicações:
 - Armazenamento e recuperação de dados.
 - Indexação.
 - Detecção de duplicatas.
 - Verificação de integridade e muito mais.



Fonte: Stack Overflow, 2022.

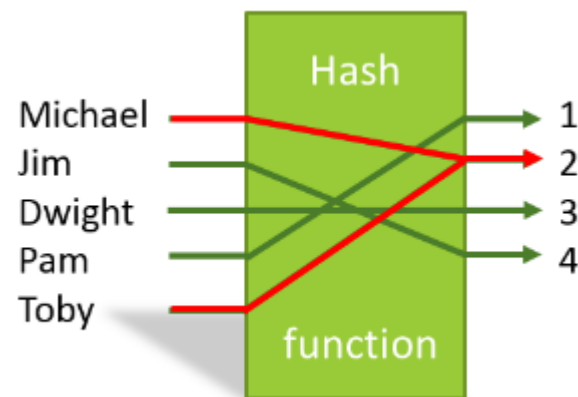
Disponível em:

<<https://stackoverflow.com/questions/39375600/what-hash-function-produces-the-maximum-number-of-collisions-when-hashing-n-keys>>.

Acessado em: 20 mar. 2024.

UNICIDADE EM FUNÇÕES HASH

- Minimiza as chances de colisões.
- Em teoria, a existência de colisões é inevitável devido ao espaço limitado de valores de *hash*.
- Funções *hash* de alta qualidade têm uma probabilidade extremamente baixa de gerar colisões em casos práticos.



Fonte: Stack Overflow, 2022.

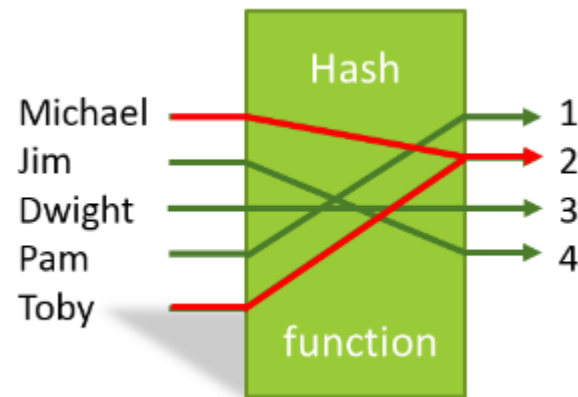
Disponível em:

<<https://stackoverflow.com/questions/39375600/what-hash-function-produces-the-maximum-number-of-collisions-when-hashing-n-keys>>.

Acessado em: 20 mar. 2024.

UNICIDADE EM FUNÇÕES *HASH*

- Quanto maior o espaço de valores de *hash*, menor a probabilidade de colisões.
- Exemplo: em criptografia de senhas, é essencial que diferentes senhas sejam convertidas em valores de *hash* diferentes.
- Mesmo que um invasor tenha acesso aos *hash* armazenados, ele terá dificuldade em determinar as senhas originais correspondentes.



Fonte: Stack Overflow, 2022.

Disponível em:

<<https://stackoverflow.com/questions/39375600/what-hash-function-produces-the-maximum-number-of-collisions-when-hashing-n-keys>>.

Acesso em: 20 mar. 2024.

INTEGRIDADE EM FUNÇÕES HASH

- Refere-se à capacidade de verificar se os dados permaneceram inalterados durante sua:
 - Transmissão.
 - Armazenamento.
 - Processamento.
- É possível detectar qualquer modificação ou corrupção nos dados.
- Garante que os dados não tenham sido alterados, fornecendo um mecanismo para validar a sua integridade.



Fonte: Kuleuven, 2020.

Disponível em: <<https://www.esat.kuleuven.be/cosic/blog/co6gc-hash-functions/>>.

Acesso em: 20 mar. 2024.



Fonte: Secureideas, 2020.
Disponível em:
<<https://www.secureideas.com/blog/2020/07/hashin-g-functions-cissp-domain-3.html>>.
Acesso em: 20 mar. 2024.

CONFUSÃO EM FUNÇÕES HASH

- Propriedade de uma função em tornar o resultado do valor de *hash* altamente dependente das mudanças nos dados de entrada.
- Uma função *hash* com uma boa confusão produzirá um valor de *hash* radicalmente diferente para entradas ligeiramente diferentes.
- Necessária para garantir que pequenas alterações nos dados de entrada causem grandes efeitos nos valores de *hash* resultantes.





Fonte: Secureideas, 2020.
Disponível em:
<<https://www.secureideas.com/blog/2020/07/hashing-functions-cissp-domain-3.html>>.
Acessado em: 20 mar. 2024.

CONFUSÃO EM FUNÇÕES HASH

- Espalha as propriedades dos dados de entrada de forma equilibrada e caótica em todo o processo de cálculo do valor de *hash*.
- Atingido por técnicas de embaralhamento que amplificam as diferenças nas entradas e propagam essas diferenças ao longo de cada etapa.
- Dificulta a manipulação ou a falsificação dos dados sem alterar drasticamente o valor de *hash* correspondente.
- Ajuda a evitar correlações entre os dados de entrada e os valores de *hash*.



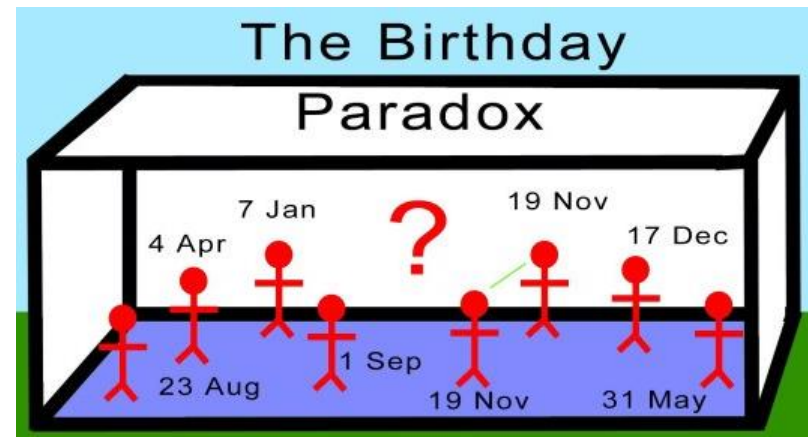
Fonte: Secureideas, 2020.
Disponível em:
<<https://www.secureideas.com/blog/2020/07/hashing-functions-cissp-domain-3.html>>.
Acesso em: 20 mar. 2024.

DIFUSÃO EM FUNÇÕES HASH

- Propriedade de uma função em espalhar as características dos dados de entrada por todo o valor de *hash* resultante.
- Garante que qualquer mudança nos dados de entrada afete significativamente todos os bits do valor de *hash*.
- Alcançada por meio do processamento iterativo dos dados de entrada em uma série de transformações.
- Garante que qualquer alteração nos dados de entrada produza um efeito significativo em todos os bits do valor de *hash*.

PARADOXO DO ANIVERSÁRIO EM FUNÇÕES *HASH*

- Fenômeno estatístico que ilustra a probabilidade de ocorrência de colisões em um conjunto de elementos.
- Aplicado para ilustrar a probabilidade de duas entradas diferentes produzirem o mesmo valor de *hash* (colisão).
- A probabilidade de colisões aumenta à medida que o número de elementos aumenta.



Fonte: Youtube, 2015.

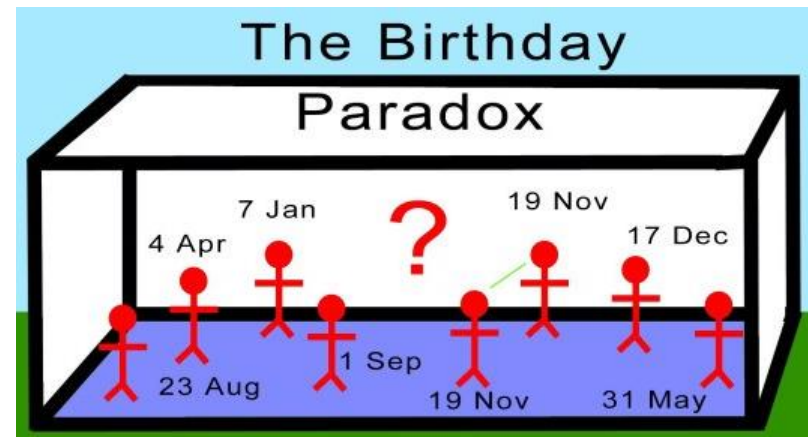
Disponível em:

<<https://www.youtube.com/watch?app=desktop&v=QrwV6fJKBi8>>.

Acesso em: 20 mar. 2024.

PARADOXO DO ANIVERSÁRIO EM FUNÇÕES HASH

- A probabilidade de colisões se torna significativa quando o número de entradas se aproxima da raiz quadrada do espaço de valores de *hash*.
- Exemplo:
 - Se o espaço de valores tiver 10.000 valores possíveis.
 - A probabilidade de pelo menos uma colisão ocorrer aumenta consideravelmente quando houver cerca de 100 entradas.

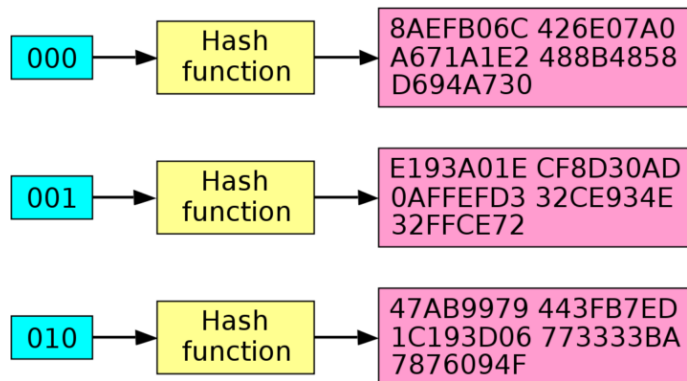


Fonte: Youtube, 2015.

Disponível em:

<<https://www.youtube.com/watch?app=desktop&v=QrwV6fJKBi8>>.

Acesso em: 20 mar. 2024.

Input**Hash sum**

Fonte: Research Gate, 2007.

Disponível em:

<https://www.researchgate.net/figure/FIGURA-33-Funcao-de-Hash-Fonte-WIKIPEDIA-2007b_fig1_237673107>.

Acesso em: 20 mar. 2024.

APLICAÇÕES DE FUNÇÕES HASH

- Criptografia.
- Armazenamento e verificação de integridade de dados.
- Tabelas de dispersão (*hash tables*).
- Verificação de integridade de arquivos e downloads.
- Verificação de integridade de mensagens e comunicações.

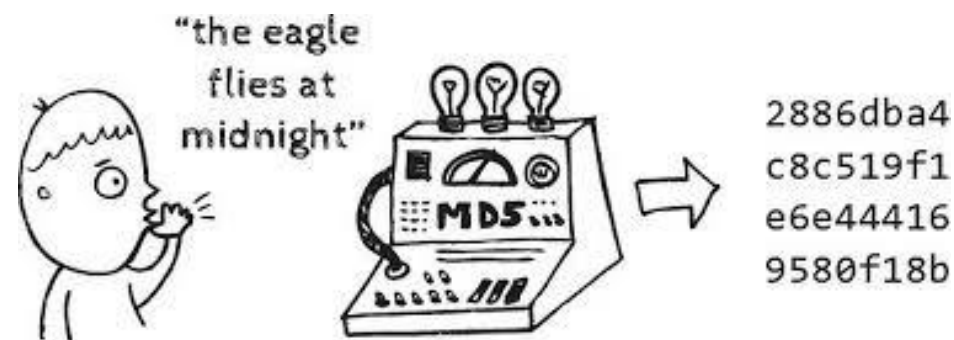
Aula 03

Funções *Hash*

Principais algoritmos de Funções *Hash* —

MD5 (MESSAGE DIGEST ALGORITHM 5)

- Opera em blocos de dados de tamanho fixo (512 bits) e utiliza operações *bitwise*, aritméticas e lógicas para processar os dados de entrada.
- Passa por quatro etapas principais: inicialização, processamento de blocos, finalização e geração do valor de *hash*.
- Devido a vulnerabilidades descobertas, o MD5 não é mais considerado seguro para aplicações criptográficas críticas.
- É possível gerar colisões de forma prática com ataques computacionais modernos.



Fonte: Crypto News, 2017.

Disponível em: <<https://www.crypto-news.net/what-are-bitcoin-hash-functions/>>.

Acesso em: 20 mar. 2024.

SHA-1 (SECURE HASH ALGORITHM 1)

- Foi amplamente utilizado, mas agora é considerado inseguro para aplicações criptográficas críticas.
- Opera em blocos de 512 bits e gera um valor de *hash* de 160 bits.
- Em 2005 foram divulgadas vulnerabilidades teóricas no SHA-1, indicando que é possível encontrar colisões.



SHA-2 (SECURE HASH ALGORITHM 2)

- Família de algoritmos de função *hash* criptográfica que foi projetada como uma melhoria do SHA-1.
- Inclui vários tamanhos de saída, como SHA-224, SHA-256, SHA-384 e SHA-512.
- Esses números representam o tamanho dos valores de *hash* produzidos pelo algoritmo: 224, 256, 384 e 512 bits.
- Quanto maior o tamanho de saída, maior é a resistência a ataques criptográficos.



SHA-3 (SECURE HASH ALGORITHM 3)

- Inclui vários tamanhos de saída, como SHA3-224, SHA3-256, SHA3-384 e SHA3-512.
- Geram valores de *hash* de 224, 256, 384 e 512 bits.
- Oferece uma maior segurança em relação ao SHA-2 e é recomendado para aplicações que exigem um alto nível de proteção.
- Amplamente utilizado em sistemas de autenticação, assinaturas digitais e sistemas de segurança em redes.



RIPEMD-160 (*RACE INTEGRITY PRIMITIVES EVALUATION MESSAGE DIGEST-160*)

- Foi projetado como uma extensão do algoritmo RIPEMD original, com um tamanho de saída de 160 bits.
- Foi desenvolvido como uma alternativa aos algoritmos SHA-1 e MD5.
- É utilizado em várias áreas, como criptografia, integridade de dados, assinaturas digitais e autenticação de arquivos.

Text	RIPEMD160 hash
hello →	108f07b8382412612c04 8d07d13f814118445acd

Fonte: Toolsyep.
Disponível em: <<https://toolsyep.com/pt/gerador-de-hash/ripemd160/>>.
Acesso em: 20 mar. 2024.

BLAKE2

- É altamente versátil, oferecendo diferentes versões com tamanhos de saída variáveis.
- Inclui *Blake2b* (com tamanho de saída de 512 bits) e *Blake2s* (com tamanho de saída de 256 bits).
- Seu desempenho é excepcionalmente rápido.
- Otimizado para processadores modernos.
- Possui propriedades de difusão e confusão bem equilibradas.

8



Você chegou ao final da Aula 03
Funções *Hash*